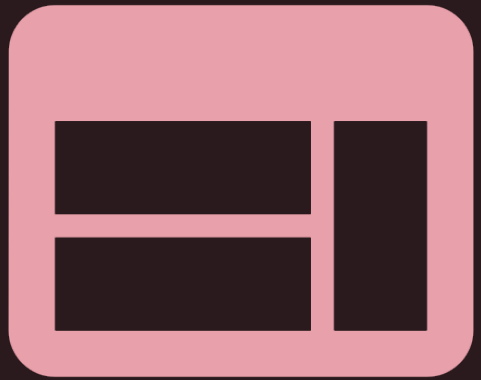
Higher National Diploma in Network Engineering, Year 2
BSc (Hons) Ethical Hacking & Network Security, Year 2

PRINCIPLES OF INFORMATION SECURITY

Web application flaws and their fixes



Day 6 - Morning

Running case: Taxila College, a fictional college

Niranga Dharmaratna
School of Computing & Engineering
National Institute of Business Management

Today

What a web application must never trust, and how it guards your session.

- The boundary between code and data - 35 min
- Injection into a database - 30 min
- Scripts in another user's browser - 30 min
- Sessions, cookies and forged requests - 30 min
- Misconfiguration and secure development - 30 min



Today serves two outcomes

LO1 Explain the fundamentals of information security.

LO2 Discuss the key characteristics of cryptography and cryptographic systems.

LO3 Analyze the principles of encryption, decryption, hashing, and data integrity.

LO4 Outline the process of ensuring digital security.

LO5 Describe the fundamentals of threats and countermeasures.

LO6 Explain the fundamentals and process of business continuity planning.

Where we start

RECALL

Authorisation decides what an authenticated user may do.

- Secure web traffic proves a server's identity and **encrypts** the channel
- The portal is a web application: anyone on the internet can reach it and type into it
- Today: what the portal must **never** trust, and how it keeps you signed in

A portal that does not trust what users type.

IT

HEAD OF IT

Whoever holds that cookie is that student.

The Head of IT, on the portal's session cookie.

CONCEPT 1

The boundary between code and data

The portal must treat what a user types as data, never as a command.

TRY IT FIRST

Input that stays data fills a blank; input read as a command changes the form.

35 min

Demonstration: data or command

1. Choose a prepared value for a blank in a fixed instruction
2. Watch it sit inside the blank, as data
3. Choose a value that runs past the blank, and watch it join the instruction
4. Switch on safe handling and see it kept inside the blank again

Injection is input that is read as a command instead of as data.

Where input lands

- Injection happens when input is read as a **command**, not as data
- The fault is in how the **application** builds the command, not in the database
- The **OWASP Top 10** is the map; today touches injection (A05), misconfiguration (A02) and authentication failures (A07)

× COMMON MISTAKE

The database is at fault for injection.

The fault is the application building a command from input.

You already know this

YOU ALREADY KNOW THIS

A form letter has one blank line to fill. Write a name on the line and the letter reads as meant. Keep writing, past the line and over the printed words, and you change what the letter tells the reader to do.

MAPPING

filling the blank with a name = data · **writing over the rest of the form =** injection

T TAXILA

The portal must treat what a user types as data, never as a command (register item 11).

CONCEPT 2

Injection into a database

A crafted login must not return every student's record.

TRY IT FIRST

A query built by joining text can be changed by its input; a parameterised query cannot.

30 min

Demonstration: a login query, built two ways

1. Build the query by joining text, and step a prepared test value through it
2. Watch part of the value become a condition that is always true
3. Switch to a parameterised query and step the same value through
4. Run the test with your index and read the record counts

Join input into a query and it can change the query. Keep it beside the query and it cannot.

Two ways to build the login query

Joined into the query text

What happens

Input is placed inside the query, so part of it can become a condition that is always true.

The result

One record was asked for; every record comes back. This is SQL injection.

A parameterised query

What happens

The query holds placeholders; the input travels beside it, as data only.

The result

Input can never change the question. The crafted login matches nothing.

Two more layers help: validate input, and give the database account only the rights the portal needs.

You already know this

YOU ALREADY KNOW THIS

A receptionist reads aloud whatever a visitor writes on a badge. Write a name and they say the name. Write a name and an order, and a receptionist who reads it all aloud has given an instruction. One who always treats the badge as a name cannot be turned.

MAPPING

a badge that says open all doors and is obeyed = injected input · **a receptionist who only ever treats the badge as a name =** a parameterised query

T TAXILA

A crafted login to the portal must not return every student's record.

CONCEPT 3

Scripts in another user's browser

A comment field on the portal must not run a visitor's script.

TRY IT FIRST

Input echoed into a page is read as markup until the page encodes it.

30 min

Demonstration: output encoding

1. Place a prepared, harmless note on a page with encoding off
2. See the note's markup take effect and change the page
3. Switch output encoding on
4. See the same note shown as plain text, with your index

Encode what the page shows, and input can only ever be displayed, never run.

Scripts in another user's browser

- **Cross-site scripting (XSS)** shows a user's input as live script in another user's browser
- Reflected scripting rides a link; stored scripting is saved and served to everyone
- It harms the **user**, where database injection harms the server
- **Encoding** the output makes input display as plain text, so it can never run

× COMMON MISTAKE

Cross-site scripting attacks the server.

It runs in another user's browser.

You already know this

YOU ALREADY KNOW THIS

A noticeboard where a helper reads aloud every note that is pinned up. Most notes are messages. But a note written as an instruction, read aloud as one, turns the board into a way to give orders. Laminate every note so it is only shown, and the trick fails.

MAPPING

a pinned note that gives instructions = an injected script · **laminating every note so it is only displayed =** encoding the output

T TAXILA

A comment field on the portal must not run a visitor's script.

CONCEPT 4

Sessions, cookies and forged requests

The portal's session cookie must carry HttpOnly, Secure and SameSite.

TRY IT FIRST

Each cookie attribute closes a different gap.

30 min

Demonstration: cookie guards

1. Start with no guards and read the three rows
2. Switch on HttpOnly; watch which row closes
3. Switch on Secure, then SameSite
4. Set SameSite to Strict and read the everyday row

Each setting closes one gap. No one setting closes all three.

After sign-in, the token is you

RECALL

On Day 5 you met authentication: the step that checks who you are.

- You give your password **once**; the portal does not ask on every page
- It creates a **session token**, a long random value, and sends it in a cookie
- Each later request carries the token, and the portal finds your session
- Whoever holds the token **is you**, until it expires

Guard the token as you would guard the password.

Three attributes, three gaps

Scripts	Connections	Other sites
HttpOnly	**Secure**	**SameSite**
Hides the cookie from every script on the page.	Sends the cookie only over an encrypted connection.	Leaves the cookie off requests another site started.
Closes a script reading the token.	*Closes the token travelling as plain text.*	*Closes a forged request carrying it.*

Set all three in one line: `Set-Cookie: session=...; HttpOnly; Secure; SameSite=Lax`

Forged requests ride your session

- You are signed in to the portal in one tab
- A page on **another site** makes your browser post a form to the portal
- The browser adds the portal's cookie, as it always does
- The portal's log shows a valid session on a request you never meant to send

× COMMON MISTAKE

The portal uses a secure connection, so its cookie is already safe.

That guards it in transit only. Scripts and forged requests still reach it.

You already know this

YOU ALREADY KNOW THIS

At an event you show your ticket once, then wear a wristband. Whoever wears it gets in. A pickpocket cannot reach a band fixed to your arm, and a good event checks it only at its own entrance.

MAPPING

the wristband = the session cookie · **the pickpocket who cannot reach it =** HttpOnly · **the gate that only accepts it at your own entrance =** SameSite

T TAXILA

The portal's session cookie must carry HttpOnly, Secure and SameSite (register item 12).

CONCEPT 5

Misconfiguration and secure development

An admin page left on its default login is an open door into the portal.

TRY IT FIRST

The OWASP Top 10, with today's items marked.

30 min

Demonstration: the OWASP map

1. Open the whole map of ten entries
2. Show today's three
3. Read where earlier days already took you

A map of the most common risks, not a list of every flaw.

Open doors need no exploit

- A **default password** is printed in the product's manual
- An **admin page** open to everyone, or left on its default login
- This is security misconfiguration: how a system is set up, not how it is written
- Fixes are plain: change defaults, hide admin pages, turn off what is not needed

YOU ALREADY KNOW THIS

A new safe comes with a factory combination, printed in its manual. Anyone with the manual can open it, and nobody has to break anything.

MAPPING

the factory combination = a default password · **changing it =** basic hardening

Secure development, in one line

- Validate input on the **server**
- Use **parameterised** queries
- **Encode** all output

No single control is enough; the layers work together.

× COMMON MISTAKE

Validation in the browser is a security control.

It runs on the user's machine and is bypassed in seconds. Validate on the server.

Common mistakes

× COMMON MISTAKE

The database is at fault for injection.

The fault is the application building a command from input.

× COMMON MISTAKE

Cross-site scripting attacks the server.

It runs in another user's browser.

× COMMON MISTAKE

Validation in the browser is a security control.

It is bypassed in seconds; validate on the server.

Before the next day

READ	**Security in Computing, sections 4.1 and 4.3** Attacks in the browser and on user and website data; then the OWASP Top 10 on its own site.
PRACTICE	**A doctor-channelling booking platform** Practice question 6: injection, scripting, cookie attributes.
PACK	**Evidence pack 6** After this afternoon's lab. Add web risks to your group's register.

Questions & Discussion

This afternoon: find the flawed lines in the portal's code, fix them, and prove the fix.



Disclaimer & Copyright

The information in this presentation is for general teaching purposes only. It is not comprehensive and is not professional advice; any reliance you place on it is at your own risk. I aim to keep it accurate, but I make no representations or warranties of any kind about its completeness, accuracy, reliability or suitability.

Security tools, standards and laws change. Versions, editions and commencement dates in this presentation were checked at the time of writing; check the current documentation before relying on them.

This presentation may refer to third-party websites or content. I do not endorse their views or information, and I am not responsible for their content.

Taxila College is fictional. Its campuses, people, systems and incidents were invented for teaching. Product and tool names, for example Kali Linux, VirtualBox, CyberChef and Wireshark, belong to their owners; this material is not affiliated with or endorsed by any of them.

Practise only on your own virtual machine, the module's provided files and application, and the portal's demos. Testing a system you do not own, without written permission, is an offence under Sri Lanka's Computer Crime Act, No. 24 of 2007.

© Niranga Dharmaratna, School of Computing & Engineering, National Institute of Business Management. Teaching content under CC BY-NC-SA 4.0; code and demonstrations under MIT; Open Sans under the SIL Open Font Licence.

Revision aad37b9